

LAB # 10

Implementation of constructors in java

Objective

Introducing string class and to explore the strings class built-in methods.

Lab Goals

- Understand the concept of constructors and how to define them.
- Understand the usage of the `this` keyword for referencing current object instances.

Introduction to Constructor In Java:

In Java, every class has its constructor that is invoked automatically when an object of the class is created. A constructor is similar to a method but in actual, it is not a method.

A Java method and Java constructor can be differentiated by its name and return type. A constructor has the same name as that of class and it does not return any value. For example,

```
class Test {  
    Test() {  
        // constructor body  
    }  
}
```

Here, Test() is a constructor. It has the same name as that of the class and doesn't have a return type.

```
class Test {  
    void Test() {  
        // method body  
    }  
}
```

Here, Test() has the same name as that of the class. However, it has a return type void. Hence, it's a method, not a constructor.

Example of Java Constructor

```
class Main {  
    private int x;  
  
    // constructor  
    private Main(){  
        System.out.println("Constructor Called");  
        x = 5;  
    }  
  
    public static void main(String[] args){  
        // constructor is called while creating object  
        Main obj = new Main();  
        System.out.println("Value of x = " + obj.x);  
    }  
}
```

Output:

```
Constructor Called  
Value of x = 5
```

In the above example, we have a private constructor named Main(). Inside the main method, we are creating an object named obj of the class.

Parameterized Constructor

Similar to methods, we can pass parameters to a constructor. Such constructors are known as a parameterized constructor. For example,

```
private Constructor (arg1, arg2, ..., argn) {  
    // constructor body  
}
```

Example of Parameterized constructor

```
class Vehicle {  
  
    int wheels;  
  
    // constructor accepting single value  
    private Vehicle(int wheels){  
        this.wheels = wheels;  
        System.out.println(wheels + " wheeler vehicle created.");  
    }  
  
    public static void main(String[] args) {  
  
        // calling the constructor by passing single value  
        Vehicle v1 = new Vehicle(2);  
        Vehicle v2 = new Vehicle(3);  
        Vehicle v3 = new Vehicle(4);  
    }  
}
```

Output:

```
2 wheeler vehicle created.  
3 wheeler vehicle created.  
4 wheeler vehicle created.
```

In the above example, we have a constructor named Vehicle(). The constructor accepts a single parameter named wheels.

Here, while creating objects, we are passing arguments to the constructor. And, based on the argument, it is generating the output.

Example of Parameterized constructor

```
class Rectangle
{
    int length;
    int breadth;
    //constructor to initialize length and breadth of rectangle
    Rectangle(int l,int b)
    {
        length = l;
        breadth= b;
    }
    //method to calculate area of rectangle
    int area()
    {
        return (length * breadth);
    }
}

//class to create rectangle objects and calculate area
class ParameterizedConstructor
{
    public static void main(String[] args)
    {
        Rectangle firstRect = new Rectangle(5,6);
        Rectangle secondRect = new Rectangle(7,8);
        System.out.println("Area of First Rectangle : " +firstRect.area());
        System.out.println("Area of Second Rectangle : "+secondRect.area());
    }
}
```

In this example, the class Rectangle contains a parameterized constructor. The specified values passed to this constructor are used to initialize instance variables length and breadth of an object. On the execution of the statement,

```
Rectangle firstRect = new Rectangle (5, 6);
```

the arguments 5 and 6 are passed to the parameters a and b of the Rectangle object's constructor which are then used to initialize length and breadth instance variables. Similarly it works for the other object secondRect.

Exercise

- A. Write a program that creates two classes one is Student another is the main method class. Student contains private fields of roll-no, semester, GPA, and use a constructor and a method to get and show the above information of three students using get and show method in java.
- B. Write a Java program to implement a Rectangle class with the following specifications:
1. The class should have two instance variables: length and width, both of type integer.
 2. Implement a constructor that takes two parameters **l** and **w** and initializes the length and width variables accordingly.
 3. Implement a method named calculateArea that calculates and returns the area of the rectangle (length * width).
 4. In the main method:
 - Create an object of the Rectangle class with length 5 and width 4.
 - Call the calculateArea method on the object and store the result.
 - Print the calculated area.